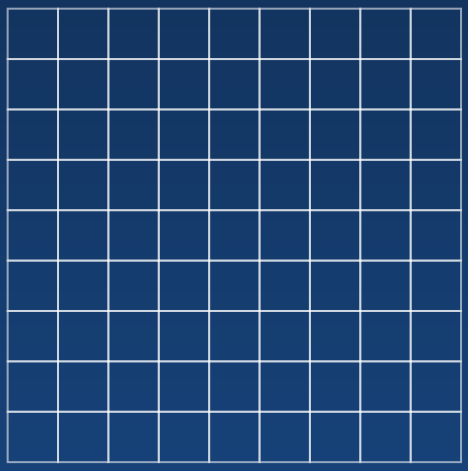
COMPUTER GRAPHICS · QUESTION BANK

LEC 1–6

بنك الأسئلة من المراجع بس

كل سؤال وإجابته منقولين من المحاضرات 1 لـ 6، والسكاشن من 1 لـ 7، و1 Exercise Sheet.

Transformations | Circle · Ellipse | DDA · Bresenham | Viewport | Color & LUT | Theory | Code



77	23	13
سؤال بإجابته	سؤال متكرر	ملف مرجع

المحتوى

01 **أسئلة Lec 1**
Lec 1 · SEC 1 and 2 · Exercise Sheet 1
10 سؤال — Q1–Q10

02 **أسئلة Lec 2**
Lec 2 · Graphics_SEC 3
14 سؤال — Q11–Q24

03 **أسئلة Lec 3**
Lec 3 · Graphics_Sec 4
11 سؤال — Q25–Q35

04 **أسئلة Lec 4**
Lec 4 · Graphics sec5
14 سؤال — Q36–Q49

05 **أسئلة Lec 5**
Lec 5 · Graphics_Sec 6
8 سؤال — Q50–Q57

06 **أسئلة Lec 6**
Lec 6 · Graphics_Sec 7
14 سؤال — Q58–Q71

07 **أسئلة الأكواد**
Lec 3 · Lec 4 · Lec 5
6 سؤال — Q72–Q77

إزاي تستخدم البنك ده

- المصادر 13 ملف بس: Lec 1 لـ Lec 6، وSEC 1 and 2، وGraphics_SEC 3، وGraphics_Sec 4، وGraphics، وsec5، وGraphics_Sec 6، وGraphics_Sec 7، وExercise Sheet 1.
- الإجابة تحت كل سؤال منقولة من المرجع زي ما هي، والسطر الصغير **Answer from** فيه الملف والصفحة.
- جه في = الملفات اللي السؤال أو فكرته جت فيها. العلامة الحمرا متكرر = جه في ملفين أو أكتر.

★ الأسئلة اللي جت أكتر (ابدأ بيها)

الجزء	السؤال	جه كام مرة
أسئلة Lec 1	**Q7** · Compare and contrast vector and raster display technologies (how each constructs pictures, file size, scalability and image quality). Which is better for a logo, a photo and an icon?	×9
أسئلة Lec 1	**Q10** · Briefly describe the main stages of the OpenGL pipeline.	×7
أسئلة Lec 1	**Q1** · Define computer graphics and list the three main operations it involves. What is the difference between computer graphics and image processing?	×4
أسئلة Lec 1	**Q5** · Describe the components of the computer graphic system and the role of the GPU in relation to the CPU and frame buffer.	×4
أسئلة Lec 1	**Q6** · What are input primitives? Give the input primitive of each input device.	×4
أسئلة Lec 1	**Q3** · Differentiate between geometric models and mathematical models in computer graphics.	×3
أسئلة Lec 1	**Q4** · Explain the difference between non-interactive and interactive computer graphics.	×3
أسئلة Lec 1	**Q8** · Describe the conceptual framework of computer graphics.	×3
أسئلة Lec 1	**Q2** · Discuss the interdisciplinary nature of computer graphics.	×2
أسئلة Lec 2	**Q11** · Describe the process of transforming coordinates from modeling to device coordinates.	×2

SECTION 01 · BASICS OF COMPUTER GRAPHICS

أسئلة Lec 1 01

Q1 **Define computer graphics and list the three main operations it involves. What is the difference between computer graphics and image processing?** متكرر

- Computer graphics are concerned with the **creation** of images.
- Image processing: analyze pictures to derive descriptions (often in mathematical or geometrical forms) of objects appeared in the pictures.
- Computer graphics means **creation, storage and manipulation** of models and images.

Answer from: Lec 1 · p6–7 · SEC 1 and 2 · p2

Exercise Sheet 1 · Q1-1 | Exercise Sheet 1 · Q3-1 | SEC 1 and 2 · p2 | Lec 1 · p6–7 | جه في

Q2 **Discuss the interdisciplinary nature of computer graphics.** متكرر

Computer graphics is a cross-disciplinary field in which physics, mathematics, human perception, human-computer interaction, engineering, graphic design, and art all play important roles.

- We use **physics** to model light and to perform simulations for animation.
- We use **mathematics** to describe shape and motion amount and direction.
- **Human perceptual abilities** determine our allocation of resources — we don't want to spend time rendering things that will not be noticed.
- We use **engineering** in optimizing the allocation of bandwidth, memory, and processor time.
- **Graphic design and art** combine with human-computer interaction to make the computer-to-human direction of communication most effective.

Answer from: Lec 1 · p7

Exercise Sheet 1 · Q3-6 | Lec 1 · p7 | جه في

Q3 **Differentiate between geometric models and mathematical models in computer graphics.** متكرر

- **Geometric model:** describes how objects appears (it defines attributes that describe color, texture or shape of the object).
- **Mathematical model:** is a model of a physical process in which it describes how object moves.

Answer from: Lec 1 · p9 · SEC 1 and 2 · p3

Exercise Sheet 1 · Q3-2 | SEC 1 and 2 · p3 | Lec 1 · p9 | جه في

Q4 **Explain the difference between non-interactive and interactive computer graphics.** متكرر

- **Non-interactive computer graphics:** the picture is produced on the monitor, and the user does not have any control over the image as shown in **animation movies**. It involves only **one-way communication** between the computer and the user where users can see the produced image, and cannot make any change in the image.
- **Interactive computer graphics:** the picture is produced on the monitor, and the user can make any change by editing modification with an input device. It involves **two-way communication** between the computer and the user as shown in **games**.

Answer from: Lec 1 · p11 · SEC 1 and 2 · p3

Exercise Sheet 1 · Q3-3 | SEC 1 and 2 · p3 | Lec 1 · p11 | جه في

Q5 Describe the components of the computer graphic system and the role of the GPU in relation to the CPU and frame buffer.

متكرر

- **I/P device** are the h/w that is used to transfer I/P to the computer. The data can be in the form of text, graphics or sound.
- **Processor (CPU & GPU):** CPU gets the specifications for a geometric shape from input device and hands the geometric data (with additional operations) to the GPU. The GPU rasterizes the data by deciding which pixels to use to represent the data, then places the pixel data into the framebuffer to show on the display screen.
- **Frame buffer:** it is a block of memory dedicated to graphics output which holds data representing all the pixels in a complete video frame.
- **O/P device** displays data from memory of the computer.

Answer from: Lec 1 · p12 · SEC 1 and 2 · p1

Exercise Sheet 1 · Q2-4 | Exercise Sheet 1 · Q3-4 | SEC 1 and 2 · p1 | Lec 1 · p12 | جه في

Q6 What are input primitives? Give the input primitive of each input device.

متكرر

Different types of data are retrieved from input devices called **input primitives**.

Device	Input primitive
Locator devices (interactive drawing)	Coordinate position (x, y)
String devices (text input)	A string of characters
Valuator (games)	Scalar values (typically between 0 and 1) — can be used to fix the length of a line, the speed of an action, or perhaps the size of a picture
Choice (interactive menu selection)	A selection from a list of options
Pick (identify a portion of a picture for further processing)	Selection of a part of the screen

Answer from: Lec 1 · p13–14 · SEC 1 and 2 · p1

Exercise Sheet 1 · Q2-3 | Exercise Sheet 1 · Q1-2 | SEC 1 and 2 · p1 | Lec 1 · p13–14 | جه في

Q7 **Compare and contrast vector and raster display technologies (how each constructs pictures, file size, scalability and image quality). Which is better for a logo, a photo and an icon?** متكرر

- **Vector displays** construct pictures by drawing straight-line segments between specified endpoint positions. Picture information is then stored as a set of line-drawing instructions. It is best for printing since it is composed of a series of mathematical curves; vector graphics print precisely even when they are enlarged. It's known as scalable vector graphics (SVG).
- **Raster displays** use a frame buffer to store the color value for each screen position (pixel). The larger the image, the more disk space the image file will take up (a 200 × 200 image requires less space than a 500 × 500 one). Scaling down these images is easy but enlarging a bitmap makes it pixelated or simply blurred.

Raster	Vector
Composed of pixels	Composed of connected sequence of points (paths)
It produces large file size	It produces small file size
It provides more details shape	It provides smooth shape but limited to set of colors
Used for details images such as photo	Used for less details images such as logos

If your project requires scalable shapes and solid colors, vector is the best choice as for logos and icons; if your project requires complex color blends, raster is the preferred format as for digital photos.

Answer from: Lec 1 · p15–16 · SEC 1 and 2 · p4

جه في Lec 1 · p15–16 SEC 1 and 2 · p4 Exercise Sheet 1 · Q3-7 Exercise Sheet 1 · Q3-8 Exercise Sheet 1 · Q1-3 Exercise Sheet 1 · Q1-4 Exercise Sheet 1 · Q2-1 Exercise Sheet 1 · Q2-2 Exercise Sheet 1 · Q2-7

Q8 Describe the conceptual framework of computer graphics.

متكرر

- Interactive computer graphics consists of two major components at two levels:
- **Hardware level:** the computer receives input from interactive devices and shows images on display devices.
- **Software level:** it involves three components — **Application program** creates, stores into and retrieves from the application model such as Word, Excel, Paint or Photoshop (handles user's input). **Application model** captures all the data or objects to be pictured on the screen, that stored in the application database. **Graphics library** or application programmer's interface (API) is a set of procedures for handling graphics such as OpenGL, Direct3D or Java3D; it is intermediary between application and display hardware (graphics system).

Answer from: Lec 1 · p17 · SEC 1 and 2 · p2

Exercise Sheet 1 · Q1-6 SEC 1 and 2 · p2 Lec 1 · p17 جه في

Q9 Compare Cartesian and display coordinates. What do glutInitWindowSize, glutInitWindowPosition and gluOrtho2D specify?

- 2D is a good stepping stone towards 3D; many issues are much easier to understand in 2D; no need to simulate lights, cameras, etc.; 2D is still important and the most common use of computer graphics.
- OpenGL stands for Open Graphics Library, producing 2D and 3D graphics; it supports every major operating system and a wide variety of programming languages, but the primary interface of OpenGL is in C++.
- **Cartesian coordinates** in math, engineering: modeled as floating point, X increasing right, Y increasing up.
- **Display (physical) coordinates:** integer values only, X increasing right, Y increasing down by 1 unit = 1 pixel.
- We specify a display window with a width of 400 pixels and a height of 300 pixels using `glutInitWindowSize(400, 300)`; `glutInitWindowPosition(50, 100)` gives an initial location for the upper left corner of the display window; then specify the region of Cartesian space that occupies the window using `gluOrtho2D(xmin, xmax, ymin, ymax)`. This region is known as the **clipping region**.
- OpenGL uses normalized device coordinates: the coordinates of each vertex should be between −1.0 and 1.0.

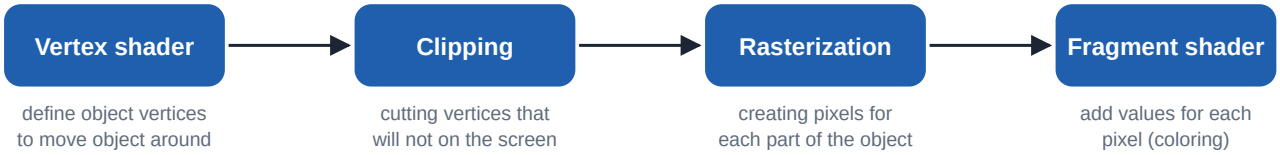
Answer from: Lec 1 · p18–21

Lec 1 · p18–21 جه في

Q10 Briefly describe the main stages of the OpenGL pipeline. متكرر

- Geometric transformations in 2D (note the z-coordinate is 0): translate, rotate, scale. Positive angles rotate counter-clockwise, here about the origin.
- Transformations can be composed (combined by multiplying matrices) but are NOT commutative, so proper order is vital.
- **Graphics pipeline** is a conceptual model that describes what steps a graphics system needs to perform to render a 3D scene to a 2D screen.

OpenGL pipeline



define object vertices to move object around

cutting vertices that will not on the screen

creating pixels for each part of the object

add values for each pixel (coloring)

Answer from: Lec 1 · p22–24 · SEC 1 and 2 · p2

Exercise Sheet 1 · Q1-5 | Exercise Sheet 1 · Q3-5 | SEC 1 and 2 · p2 | Lec 1 · p22–24 | جه في

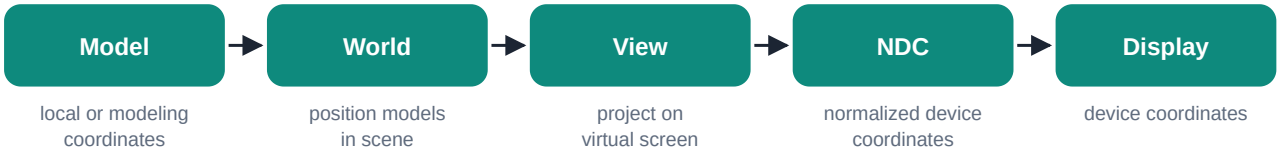
Exercise Sheet 1 · Q2-8 | Exercise Sheet 1 · Q2-6 | Exercise Sheet 1 · Q2-5

SECTION 02 · RASTER IMAGES, FRAME BUFFER, COLOR

أسئلة Lec 2 02

Q11 **Describe the process of transforming coordinates from modeling to device coordinates.** متكرر

Coordinates and transformations



1. Object shapes are defined within their own modeling-coordinate systems.
2. These shapes are then placed within a global world-coordinate scene.
3. The world-coordinate data is processed through the viewing pipeline, converting it into viewing and projection coordinates, and further into normalized coordinates.
4. Finally, device drivers take the normalized-coordinate representation of the scene and send it to the output devices for display.

Answer from: Lec 2 · p3–9 · Graphics_SEC 3 · p1

Graphics_SEC 3 · p1 Lec 2 · p3–9 جه في

Q12 What are the elements (output primitives) of pictures created in computer graphics and their attributes?

- **Output primitives:** the basic geometric structure used to describe scenes — polyline, text, filled regions, raster images. Each element has a set of attributes: the characteristics that affect how it appears, such as color, style, and thickness.
- **Polylines:** connected sequence of straight lines; pictures made up of polylines (line drawing); each line is called an edge; two adjacent lines meet at a vertex. A special case is a line segment that shrinks to a single point: `drawDot(x1, y1)`.
- Attributes of lines and polylines: color, thickness, type (dashed, solid, dotted), way edges are blended (non-joint, mitered, rounded, beveled).
- **Text:** `drawString(x, y, string)`; attributes: font face, weight, size, style, spacing, orientation.
- **Filled region:** shape filled with color; boundary of the filled shape is a polygon: `fillPolygon(poly, pattern)`; attributes: enclosing border, pattern, color.

Answer from: Lec 2 · p10–17

Lec 2 · p10–17 جه في

Q13 Define polygon and its types: simple, complex, concave and convex. متكرر

- Polygon is a closed polyline where its start point equals its last point.
- **Simple polygon** has well-defined two regions (inside and outside).
- **Complex polygon** has self-intersecting sides.
- **Concave polygon** has at least one diagonal lying outside the polygon.
- **Convex polygon** has all its diagonals lying inside the polygon.

Answer from: Lec 2 · p12 · Graphics_SEC 3 · p1

Graphics_SEC 3 · p1 Lec 2 · p12 جه في

Q14 **How are raster images stored and created? Explain monochrome, gray-scale and color raster images.** متكرر

- Image is made up of many cells (pixels) in different shades; stored as a 2D array of numerical values; the array as a whole is often called a "pixel map" or "bitmap".
- How raster images are created: **hand designed** images (paint program), **computed** images (an algorithm renders a scene), **scanned** images.
- **Monochrome** raster images (bi-level): pixels contain one of two values.
- **Gray-scale**: pixel depth = number of bits needed to represent gray level; N bits has 2^n values (2 bits → 4, 4 bits → 16, 8 bits → 256 gray levels).
- **Color** raster images: pixel contains a triple of red, green and blue; color depth of 3 allows one bit for each color; true color image has a color depth of 24 bits (8 bits for each color).

Answer from: Lec 2 · p18–24 · Graphics_SEC 3 · p1

Graphics_SEC 3 · p1 Lec 2 · p18–24 جه في

Q15 **Explain raster displays, emissive and non-emissive flat panels, and frame rate vs refresh rate.**

- **Framebuffer** is a portion of memory containing data representing all the pixels in a complete video frame. **Pixel:** one dot of the raster. **Scan line:** a row of pixels.
- **Emissive displays** convert electrical energy into light (LED). **Non-emissive displays** use optical effects to convert light from some other source into graphics patterns (LCD).
- **Frame rate (FPS):** how many unique, complete images your GPU can create and send to the monitor each second. **Refresh rate (Hz):** how many times per second your monitor can physically update the image it displays.
- GPU frame rate < monitor refresh rate → the monitor displays the old frame again → **stuttering**. GPU frame rate > refresh rate → **screen tearing**. Therefore, you should match the two features.
- **Raster scan system:** the scan controller causes the frame buffer to "send" each pixel through a converter to the appropriate physical spot; the converter takes a pixel value such as 01001011 and converts it to the corresponding quantity that produces a spot of color.

Answer from: Lec 2 · p25–30

Lec 2 · p25–30 جه في

Q16 How does a video monitor produce colors with bit planes?

For a simple color, frame buffer is implemented with 3 bit planes, one for each primary color (red, green, blue); each bit plane drives an individual color gun. For better resolution, additional bit planes can be used for each color: $2^2 \times 2^2 \times 2^2 = 2^6 = 64$ different colors.

Red	Green	Blue	Color
0	0	0	Black
0	0	1	Blue
0	1	0	Green
0	1	1	Cyan
1	0	0	Red
1	0	1	Magenta
1	1	0	Yellow
1	1	1	White

Answer from: Lec 2 · p33–34

Lec 2 · p33–34 جه في

Q17 Explain pixel color representation: direct coding and indexed color with a look-up table (LUT). متكرر

- **Color palette** is the set of possible colors displayed by the system.
- **1. Direct coding** represents each pixel's color directly. Frame buffer with 6 planes (2 for each color) → 4 levels each → $2^6 = 64$ colors. True color (24 planes, 8 for each color) → $2^{24} = 16$ million colors.
- **2. Indexed color and LookUp Table (LUT)** offers a programmable association between pixel value and final color. LUT is a compromise between our desire to have a lower storage requirement and our need to support enough number of colors. Each image pixel does not contain the full specification of its color, but only its index into the palette.
- `setPalette(39, 17, 25, 4)` then `drawDot(479, 532, 39)`. LUT of $(2^8 \times 24)/8 = 768$ bytes.

Answer from: Lec 2 · p35–38 · Graphics_SEC 3 · p1

Graphics_SEC 3 · p1 Lec 2 · p35–38 جه في

Q18 **If we use direct coding of RGB value with 3 bits per primary color, how many possible colors do we have for each pixel?**

$2^3 \times 2^3 \times 2^3 =$ **512** color

Answer from: Lec 2 · p39

Lec 2 · p39 جه في

Q19 **If we use 10 bits pixel value in a 24-bit lookup table representation, how many entries does the lookup table have and how much is the size of the lookup table in bytes?**

lookup table entries = $2^{10} =$ **1024** entry · lookup table size = $(2^{10} \times 24)/8 =$ **3072 byte**

Answer from: Lec 2 · p39

Lec 2 · p39 جه في

Q20 **If we use direct coding of RGB value with 5 bits per primary color, how many possible colors do we have for each pixel?**

Levels of red = green = blue = $2^5 = 32$ · Total colors = $32 \times 32 \times 32 =$ **32,768** colors

Answer from: Graphics_SEC 3 · p1

Graphics_SEC 3 · p1 جه في

Q21 **If we use a 10-bit pixel value in a 32-bit lookup table representation, how many entries does the lookup table have and how much is the size of the lookup table in bytes?**

Lookup table entries = $2^{10} =$ **1,024** entries · size = $(2^{10} \times 32)/8 =$ **4,096 bytes** (or 4 KB)

Answer from: Graphics_SEC 3 · p1

Graphics_SEC 3 · p1 جه في

Q22 **A frame buffer supports 12 planes of memory (4 planes for each color). Calculate the total number of colors in the palette.**

Levels of red = green = blue = $2^4 = 16$ · Total colors = 16 × 16 × 16 = **4,096** colors

Answer from: Graphics_SEC 3 · p1

Graphics_SEC 3 · p1 جه في

Q23 **Compare System A (32 bits/pixel frame buffer, no LUT) and System B (12 bits/pixel frame buffer with a 30-bit LUT) in terms of memory requirements for a 1920 × 1080 display.**

- **System A:** frame buffer = 1920 × 1080 × 32 bits = 66,355,200 / 8 = 8,294,400 bytes ≈ **8.29 MB**.
- **System B:** frame buffer = 1920 × 1080 × 12 bits = 24,883,200 bits = 3,110,400 bytes ≈ 3.11 MB · LUT entries = 2^{12} = 4,096 · LUT size = (2^{12} × 30)/8 = 15,360 bytes ≈ 0.015 MB · total ≈ 3.11 + 0.015 = **3.125 MB**.

Answer from: Graphics_SEC 3 · p2

Graphics_SEC 3 · p2 جه في

Q24 **A display system uses a frame buffer with 10 bits per pixel and a LUT with 36-bit entries; the display resolution is 2560 × 1440. (a) How many colors can be displayed simultaneously? (b) How many total colors are available in the palette? (c) Total memory (frame buffer + LUT) in megabytes? (d) Frame buffer memory with direct coding of 10 bits per primary color?**

- (a) Simultaneous colors: 10-bit pixel value → 2^{10} = **1,024** colors.
- (b) Total palette colors: 36-bit LUT → 2^{36} = **68,719,476,736** colors (≈ 68.7 billion).
- (c) Frame buffer = 2560 × 1440 × 10 bits = 36,864,000 bits = 4,608,000 bytes ≈ 4.39 MB · LUT = (2^{10} × 36)/8 = 4,608 bytes ≈ 0.0044 MB · total ≈ **4.394 MB**.
- (d) Direct coding with 10 bits per primary: 30 bits/pixel → 2560 × 1440 × 30 bits = 110,592,000 bits = 13,824,000 bytes ≈ **13.18 MB**.

Answer from: Graphics_SEC 3 · p2

Graphics_SEC 3 · p2 جه في

SECTION 03 · WINDOW TO VIEWPORT

أسئلة Lec 3 03

Q25 **Explain the difference between a window and a viewport in computer graphics. Why is the window-to-viewport transformation necessary for device independence?** متكرر

- The two-dimensional viewing transformation maps world-coordinate scenes to device coordinates, often referred to as the **window-to-viewport transformation**. This process primarily involves converting clipping-window coordinates to viewport coordinates.
- Once a world-coordinate scene has been constructed, we could set up a separate two-dimensional, viewing-coordinate reference frame for specifying the clipping window.
- To ensure device independence, graphics systems convert object descriptions into normalized coordinates (ranging from 0 to 1 or −1 to 1).
- **Window to viewport mapping** is the process of mapping or transforming a 2D world coordinate scene to device coordinates. **Window is what to be displayed; viewport is where to be displayed.**
- A **window** is a rectangular region in world coordinates that defines what part of the scene is selected for display. Defined by: xwmin, xwmax, ywmin, ywmax.
- A **viewport** is a rectangular region on the display device that defines where on the screen the selected scene is drawn. Defined by: xvmin, xvmax, yvmin, yvmax.
- The window-to-viewport transformation is necessary for device independence as the same scene description can be rendered on a 640×480 monitor, a 4K display, or a printer by simply changing the viewport — no change to the model or window is needed. Also different viewports on the same screen can show different windows of the same scene (e.g., a CAD program showing top, front, and side views).

Answer from: Lec 3 · p2–3 · Graphics_Sec 4 · p1

Graphics_Sec 4 · p1 Lec 3 · p2–3 جه في

Q26 Derive the window to viewport mapping equations. متكرر

Suppose we have a point (XW, YW) and we want to find the corresponding viewport point (XV, YV). If the relative position is the same:

(XW − XWmin)/(XWmax − XWmin) = (XV − XVmin)/(XVmax − XVmin) · (YW − YWmin)/(YWmax − YWmin) = (YV − YVmin)/(YVmax − YVmin)

Sx = (XVmax − XVmin)/(XWmax − XWmin) · Sy = (YVmax − YVmin)/(YWmax − YWmin)

XV = XVmin + Sx·(XW − XWmin) · YV = YVmin + Sy·(YW − YWmin)

Answer from: Lec 3 · p5 · Graphics_Sec 4 · p1

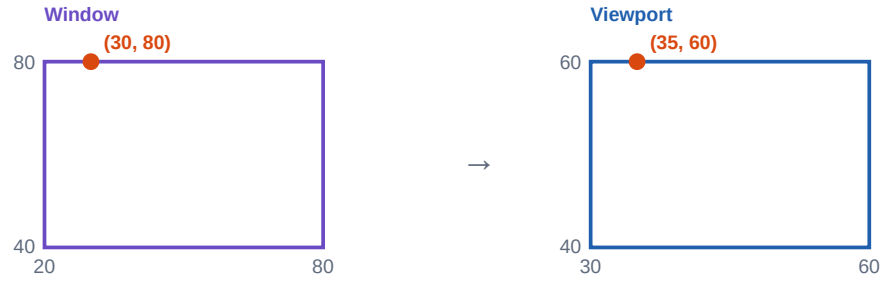
Graphics_Sec 4 · p1 Lec 3 · p5 جه في

Q27 Consider the window and viewport. The window has (W.l, W.r, W.b, W.t) = (20, 80, 40, 80) and the viewport has (V.l, V.r, V.b, V.t) = (30, 60, 40, 60). For (XW, YW) = (30, 80) find (XV, YV).

XWmin = 20, XWmax = 80, YWmin = 40, YWmax = 80 · XVmin = 30, XVmax = 60, YVmin = 40, YVmax = 60

Sx = (60 − 30)/(80 − 20) = 0.5 · Sy = 0.5

XV = 30 + (0.5·(30 − 20)) = **35** · YV = 40 + (0.5·(80 − 40)) = **60**



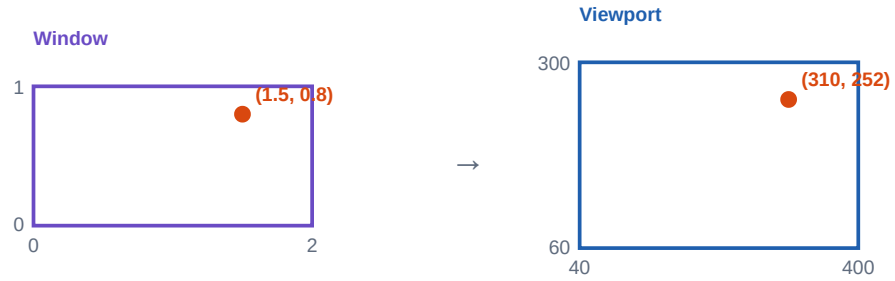
Answer from: Lec 3 · p6

Lec 3 · p6 جه في

Q28 **The window has (W.l, W.r, W.b, W.t) = (0, 2, 0, 1) and the viewport has (V.l, V.r, V.b, V.t) = (40, 400, 60, 300). For (XW, YW) = (1.5, 0.8) find (XV, YV).**

Sx = (400 − 40)/(2 − 0) = 180 · Sy = (300 − 60)/(1 − 0) = 240

XV = 40 + (180·(1.5 − 0)) = **310** · YV = 60 + (240·(0.8 − 0)) = **252**



Answer from: Lec 3 · p9

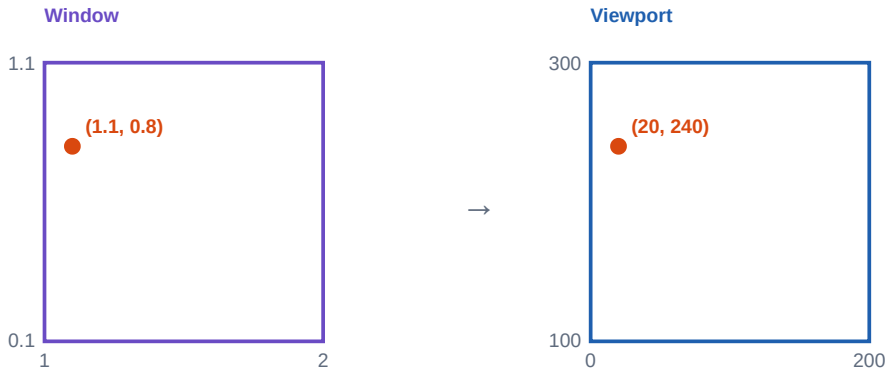
Lec 3 · p9 جه في

Q29 **Suppose a window is defined by 1.0 ≤ x ≤ 2.0, 0.1 ≤ y ≤ 1.1, and the viewport is 0 ≤ X ≤ 200, 100 ≤ Y ≤ 300. Find the viewport coordinates corresponding to the world point (1.1, 0.8).**

XWmin = 1.0, XWmax = 2.0 · YWmin = 0.1, YWmax = 1.1 · XVmin = 0, XVmax = 200 · YVmin = 100, YVmax = 300

XV = XVmin + Sx·(XW − XWmin) · YV = YVmin + Sy·(YW − YWmin)

So (XV, YV) = **(20, 240)**



Answer from: Graphics_Sec 4 · p1

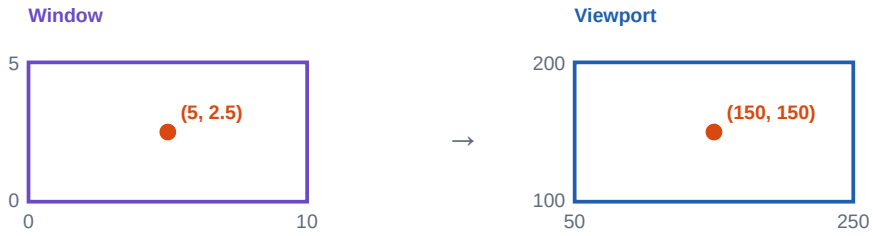
Graphics_Sec 4 · p1 جه في

Q30 **Given a window (W.l, W.r, W.b, W.t) = (0, 10, 0, 5) and a viewport (V.l, V.r, V.b, V.t) = (50, 250, 100, 200). If a point in the viewport is at (XV, YV) = (150, 150), what are its corresponding world coordinates (XW, YW)?**

XWmin = 0, XWmax = 10 · YWmin = 0, YWmax = 5 · XVmin = 50, XVmax = 250 · YVmin = 100, YVmax = 200

XV = XVmin + Sx·(XW − XWmin) · YV = YVmin + Sy·(YW − YWmin)

So (XW, YW) = **(5, 2.5)**



Answer from: Graphics_Sec 4 · p2

Graphics_Sec 4 · p2 جه في

Q31 **Find the matching viewport for a world window with aspect ratio R = 1.6 when the screen window has width 360 and height 200.**

R = 1.6 · Ww/Hh = 360/200 = 1.8 · R < Ww/Hw → Hv = H = 200 and Wv = H·R = 200 × 1.6 = 320

→ `glViewport(0,0,320,200)`



Answer from: Lec 3 · p26

Lec 3 · p26 جه في

Q32 **Find the matching viewport for a window with aspect ratio R = 2 when the screen window has width 360 and height 200.**

R = 2 · Wv/Hv = 360/200 = 1.8 · R > Wv/Hv → Wv = W = 360 and Hv = W/R = 360/2 = 180 → `glViewport(0,0,360,180)`



Answer from: Lec 3 · p27

Lec 3 · p27 جه في

Q33 **Find the matching viewport for a world window with aspect ratio R = 1.5 when the screen window has width 400 and height 300.**

World window aspect ratio R = 1.5 · screen aspect ratio = 400/300 = 1.333 → VW = 400, VH = W/R = 400/1.5 = **267**



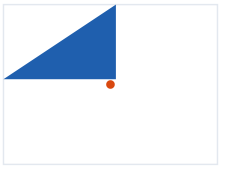
Answer from: Graphics_Sec 4 · p2

Graphics_Sec 4 · p2 جه في

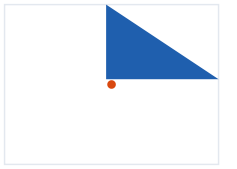
Q34 **What is drawn with each setting of gluOrtho2D for the lecture's triangle? Can you draw a simple figure without gluOrtho2D?**

`glutInitWindowSize(400, 300)` with `gluOrtho2D(-200, 200, -150, 150)` puts (0, 0) in the centre of the window. The lecture changes the order of the numbers in gluOrtho2D and shows the result:

`(-200,200,-150,150)`



`(200,-200,-150,150)`



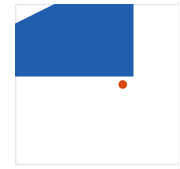
`(-200,200,150,-150)`



`(200,-200,150,-150)`



`(-100,100,-100`



Can I draw a simple figure without gluOrtho2D? With `//gluOrtho2D(-100,100,-100,100);` commented out, the triangle (−200, 10), (10, 10), (10, 150) is not displayed, while the triangle (−0.5, 0), (0, 0.5), (0.5, 0) is displayed.

Answer from: Lec 3 · p10–17

Lec 3 · p10–17 جه في

Q35 How do you set the window and viewport automatically to preserve the aspect ratio?

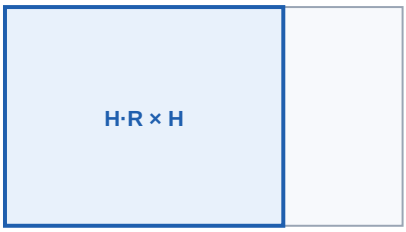
متكرر

- **Setting window:** often the programmer does not know where or how big the object of interest lies in world coordinates. The usual approach is to find a window that includes the entire object: the object's **extent (bounding box)** must be found — the aligned rectangle that just covers it.
- **Automatic setting of the viewport to preserve aspect ratio:** to draw the largest undistorted version of a figure that will fit in the screen window, specify a viewport that has the same aspect ratio as the world window. R = aspect ratio of world.
- If R > W/H: the viewport will have width W and height W/R → `glViewport(0, 0, W, W/R)`.
- If R < W/H: the viewport will have width H·R and height H → `glViewport(0, 0, H*R, H)`.

Largest undistorted viewport (screen window W × H, world aspect ratio R)



R > W/H → glViewport(0,0,W,W/R)



R < W/H → glViewport(0,0,H*R,H)

Example	Answer	Source
R = 1.6, screen window width 360, height 200	W/H = 360/200 = 1.8 → R < W/H → Hv = H = 200, Wv = H·R = 200 × 1.6 = 320 → `glViewport(0,0,320,200)`	Lec 3 · p26
R = 2, screen window width 360, height 200	W/H = 1.8 → R > W/H → Wv = W = 360, Hv = W/R = 360/2 = 180 → `glViewport(0,0,360,180)`	Lec 3 · p27
R = 1.5, screen window width 400, height 300	Screen aspect ratio = 400/300 = 1.333 → VW = 400, VH = W/R = 400/1.5 = 267	Graphics_Sec 4 · p2







Answer from: Lec 3 · p21–27 · Graphics_Sec 4 · p2

جه في Lec 3 · p21–27 Graphics_Sec 4 · p2

SECTION 04 · DDA AND BRESENHAM

أسئلة Lec 4 04

Q36 **How is a straight line displayed on a raster monitor? What are the trivial situations that do not need a line-drawing algorithm?**

- Three basic elements: **points**, **lines**, **vectors**. Points are associated with locations in space; the only characteristic that distinguishes one point from another is its position. A line is a one-dimensional shape. Vectors have magnitude and direction; they represent displacements between points, or directions.
- `setPixel` stores the current color setting into the frame buffer at integer position (x, y); `getPixel` retrieves the current frame-buffer setting for a pixel location.
- To display the line on a raster monitor, the graphics system must first project the endpoints to integer screen coordinates and determine the nearest pixel positions along the line path between the two endpoints; then the line color is loaded into the frame buffer; reading from the frame buffer, the video controller plots the screen pixels.
- Idea: a line is sampled at unit intervals in one coordinate and the corresponding integer values nearest the line path are determined for the other coordinate.
- Trivial situations (do not need line-drawing algorithms): m = 0 horizontal line; m = ∞ vertical line; m = ±1 line y = ±x.
- y = m·x + b, m = (yend − y0)/(xend − x0), b = y0 − m·x0. It works for lines with a slope of 1 or less, but doesn't work well for lines with slope greater than 1. Solution: use symmetry.

Answer from: Lec 4 · p3–13

Lec 4 · p3–13 جه في

Q37 Explain the DDA algorithm and give its advantages and disadvantages. متكرر

- The digital differential analyser (DDA) is a scan-conversion line algorithm based on using δx or δy.
- If m ≤ 1 (dx > dy): sample at unit x intervals (dx = 1): $x_{k+1} = x_k + 1$, $y_{k+1} = y_k + m$; round y to nearest integer value.
- If m > 1 (dy > dx): sample at unit y intervals (dy = 1): $y_{k+1} = y_k + 1$, $x_{k+1} = x_k + 1/m$; round x to nearest integer value.
- **Advantages:** it is a simple algorithm to implement; it is a faster algorithm than the direct line equation.
- **Disadvantages:** the floating-point arithmetic implementation of the DDA is time-consuming; the method of round-off is also time-consuming; the accumulation of round-off error in successive additions of the floating-point increment can cause the calculated pixel positions to drift away from the true line path for long line segments.

Answer from: Lec 4 · p15–17, p24 · Graphics sec5 · p1

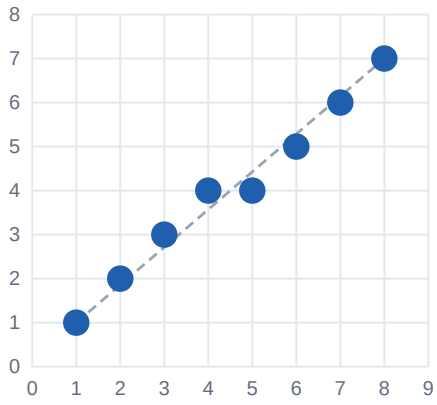
Graphics sec5 · p1 Lec 4 · p15–17, p24 جه في

Q38 Use the DDA algorithm to draw a line from (1, 1) to (8, 7).

X0 = 1, xEnd = 8, y0 = 1, yEnd = 7 · dx > dy · $x_{k+1} = x_k + 1$ · $y_{k+1} = y_k + m$

X	Y	(X, Y)
1	1	(1,1)
2	1.86	(2,2)
3	2.7	(3,3)
4	3.57	(4,4)
5	4.43	(5,4)
6	5.29	(6,5)
7	6.14	(7,6)
8	7	(8,7)

Plotted pixels



Answer from: Lec 4 · p20

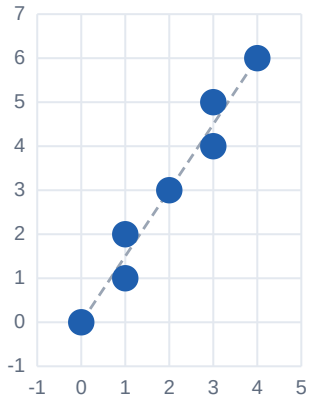
Lec 4 · p20 جه في

Q39 Use the DDA algorithm to draw a line from (0, 0) to (4, 6).

X0 = 0, xEnd = 4, y0 = 0, yEnd = 6 · dy > dx · $y_{k+1} = y_k + 1$ · $x_{k+1} = x_k + 1/m$

X	Y	(X, Y)
0	0	(0,0)
0.6	1	(1,1)
1.2	2	(1,2)
1.8	3	(2,3)
2.5	4	(3,4)
3.3	5	(3,5)
3.9	6	(4,6)

Plotted pixels



Answer from: Lec 4 · p21

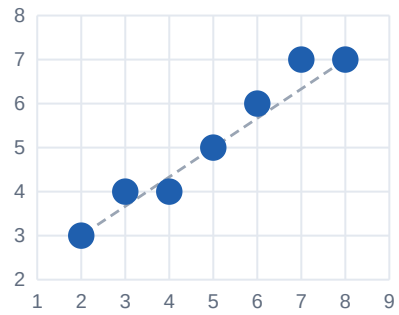
Lec 4 · p21 جه في

Q40 Use the DDA algorithm to draw a line from (9, 8) to (2, 3).

X0 = 9, xEnd = 2, y0 = 8, yEnd = 3 · m ≤ 1 · Steps = dx = (2 − 9) = −7 · k in range from 0 to 6 · $x_{k+1} = x_k - 1$ · $y_{k+1} = y_k - m$

K	x	y	(x, y)
0	8	7.2	(8,7)
1	7	6.5	(7,7)
2	6	5.8	(6,6)
3	5	5.1	(5,5)
4	4	4.4	(4,4)
5	3	3.7	(3,4)
6	2	3	(2,3)

Plotted pixels



Answer from: Lec 4 · p23

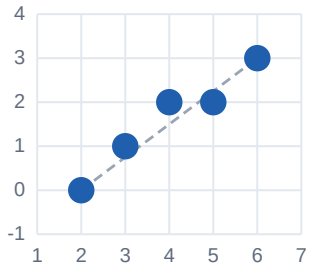
Lec 4 · p23 جه في

Q41 Use the DDA algorithm to draw a line from A(2, 0) to B(6, 3).

$|\Delta x| = |6 - 2| = 4$, $|\Delta y| = |3 - 0| = 3 \rightarrow |\Delta x| > |\Delta y| \cdot x_{k+1} = x_k + 1$, $y_{k+1} = y_k + m \cdot m = 0.75$

x	y	Setpixel(x, y)
2	0	(2,0)
3	0.75	(3,1)
4	1.5	(4,2)
5	2.25	(5,2)
6	3	(6,3)

Plotted pixels



Answer from: Graphics sec5 · p1

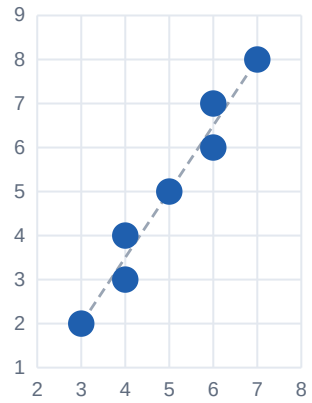
Graphics sec5 · p1 جه في

Q42 Use the DDA algorithm to draw a line from (3, 2) to (7, 8).

$|\Delta x| = |7 - 3| = 4$, $|\Delta y| = |8 - 2| = 6 \rightarrow |\Delta y| > |\Delta x| \cdot x_{k+1} = x_k + 1/m$, $y_{k+1} = y_k + 1 \cdot 1/m = 0.667$

x	y	Setpixel(x, y)
3	2	(3,2)
3.667	3	(4,3)
4.33	4	(4,4)
5	5	(5,5)
5.66	6	(6,6)
6.33	7	(6,7)
7	8	(7,8)

Plotted pixels



Answer from: Graphics sec5 · p1

Graphics sec5 · p1 جه في

Q43 **Explain Bresenham's line drawing algorithm with its advantages and disadvantages.** متكرر

- Accurate and efficient; uses only incremental integer calculations. The method is described for a line segment with a positive slope less than one, so we work by increment x and calculate y. The method generalizes to line segments of other slopes by considering the symmetry between the various octants and quadrants of the xy plane.

1. Input the two line endpoints and store the left endpoint in (x0, y0).
2. Load (x0, y0) into the frame buffer; that is, plot the first point.
3. Calculate constants Δx, Δy, 2Δy, and 2Δy − 2Δx, and obtain the starting value for the decision parameter: **p0 = 2Δy − Δx**.
4. At each x_k along the line, starting at k = 0: if $p_k < 0$, the next point to plot is $(x_k + 1, y_k)$ and $\mathbf{p_{k+1} = p_k + 2\Delta y}$; otherwise, the next point to plot is $(x_k + 1, y_k + 1)$ and $\mathbf{p_{k+1} = p_k + 2\Delta y - 2\Delta x}$.
5. Repeat step 4 Δx − 1 times.

- **Advantages (sec5):** significantly faster due to integer-only operations; more accurate and efficient, without round-off errors.
- **Disadvantages (sec5):** a full implementation must handle all combinations of slope and endpoint order by adjusting step sizes and interchanging X and Y coordinates, adding complexity.
- For |Δy| > |Δx|: swap x points with y points (sec5: "requiring coordinate swapping"); for negative slopes: coordinate decrements.

Answer from: Lec 4 · p25–32 · Graphics sec5 · p1

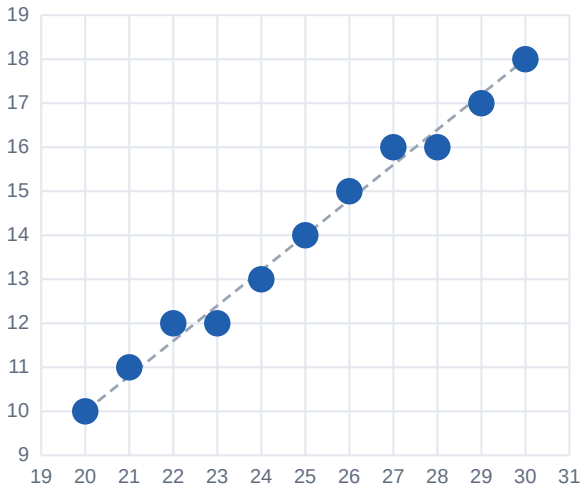
Graphics sec5 · p1 Lec 4 · p25–32 جه في

Q44 Using Bresenham's algorithm, digitize the line (20, 10) to (30, 18).

$|\Delta x| = |30 - 20| = 10$, $|\Delta y| = |18 - 10| = 8 \cdot |\Delta x| > |\Delta y| \cdot p0 = 2\Delta y - \Delta x = 16 - 10 = 6 \cdot P_k \geq 0$: x and y incremented, $P_{k+1} = P_k - 4 \cdot P_k < 0$: x incremented, y remains, $P_{k+1} = P_k + 16$

P	X	Y	SetPixel(x, y)
-	20	10	(20,10)
6	21	11	(21,11)
2	22	12	(22,12)
-2	23	12	(23,12)
14	24	13	(24,13)
10	25	14	(25,14)
6	26	15	(26,15)
2	27	16	(27,16)
-2	28	16	(28,16)
14	29	17	(29,17)
10	30	18	(30,18)

Plotted pixels



Answer from: Lec 4 · p30

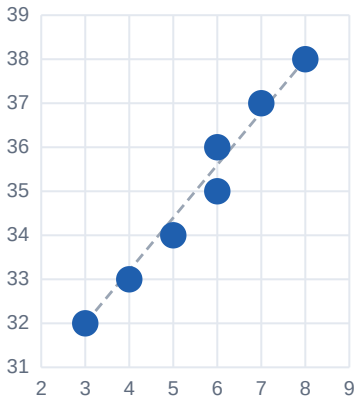
Lec 4 · p30 جه في

Q45 **Using Bresenham's algorithm, digitize the line (3, 32) to (8, 38).**

$|\Delta x| = |8 - 3| = 5$, $|\Delta y| = |38 - 32| = 6 \cdot |\Delta y| > |\Delta x| \cdot$ Swap x points with y points: X0 = 32, xend = 38 & y0 = 3, yend = 8 $\cdot |\Delta x| = 6$, $|\Delta y| = 5 \cdot p0 = 2\Delta y - \Delta x = 10 - 6 = 4 \cdot P_k \geq 0$: $P_{k+1} = P_k - 2 \cdot P_k < 0$: $P_{k+1} = P_k + 10$

P	X	Y	SetPixel(y, x)
-	32	3	(3,32)
4	33	4	(4,33)
2	34	5	(5,34)
0	35	6	(6,35)
-2	36	6	(6,36)
8	37	7	(7,37)
-	38	8	(8,38)

Plotted pixels



Answer from: Lec 4 · p33

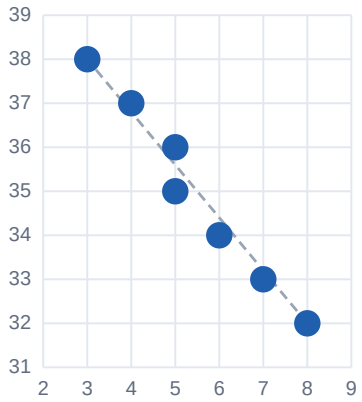
Lec 4 · p33 جه في

Q46 Using Bresenham's algorithm, digitize the line (8, 32) to (3, 38).

$|\Delta x| = |3 - 8| = |-5|$, $|\Delta y| = |38 - 32| = 6 \cdot |\Delta y| > |\Delta x| \cdot$ Swap: X0 = 32, xend = 38 & y0 = 8, yend = 3 $\cdot$ p0 = $2\Delta y - \Delta x = 10 - 6 = 4 \cdot P_k < 0$: X incremented and Y remain, $P_{k+1} = P_k + 10 \cdot P_k \geq 0$: X incremented and Y decremented, $P_{k+1} = P_k - 2$

P	X	Y	SetPixel(y, x)
-	32	8	(8,32)
4	33	7	(7,33)
2	34	6	(6,34)
0	35	5	(5,35)
-2	36	5	(5,36)
8	37	4	(4,37)
-	38	3	(3,38)

Plotted pixels



Answer from: Lec 4 · p34

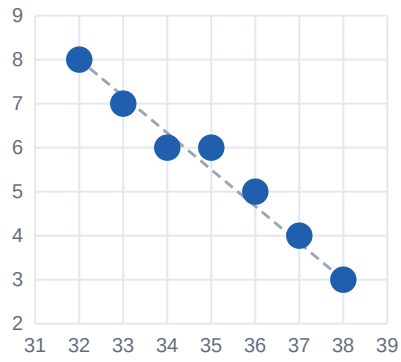
Lec 4 · p34 جه في

Q47 Using Bresenham's algorithm, digitize the line (38, 3) to (32, 8).

$|\Delta x| = |32 - 38| = 6$, $|\Delta y| = |8 - 3| = 5 \cdot |\Delta x| > |\Delta y| \cdot$ p0 = $2\Delta y - \Delta x = 10 - 6 = 4 \cdot P_k \geq 0$: X is decremented, Y is increment, $P_{k+1} = P_k - 2 \cdot P_k < 0$: X is decremented, Y is remain, $P_{k+1} = P_k + 10$

P	X	Y	SetPixel(x, y)
-	38	3	(38,3)
4	37	4	(37,4)
2	36	5	(36,5)
0	35	6	(35,6)
-2	34	6	(34,6)
8	33	7	(33,7)
-	32	8	(38,8)

Plotted pixels



Answer from: Lec 4 · p35

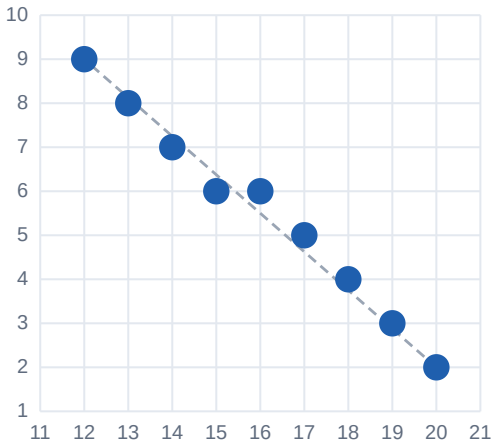
Lec 4 · p35 جه في

Q48 Using Bresenham's algorithm, digitize the line (20, 2) to (12, 9).

$|\Delta x| = |12 - 20| = |-8|$, $|\Delta y| = |9 - 2| = |7|$ · $|\Delta x| > |\Delta y|$ · $p0 = 2\Delta y - \Delta x = 14 - 8 = 6$ · If $p_k < 0$: Setpixel($x_k - 1$, y_k), $p_{k+1} = p_k + 14$ · Otherwise: Setpixel($x_k - 1$, $y_k + 1$), $p_{k+1} = p_k - 2$

P	x	y	Setpixel(x, y)
-	20	2	(20,2)
6	19	3	(19,3)
4	18	4	(18,4)
2	17	5	(17,5)
0	16	6	(16,6)
-2	15	6	(15,6)
12	14	7	(14,7)
10	13	8	(13,8)
8	12	9	(12,9)

Plotted pixels



Answer from: Graphics sec5 · p2

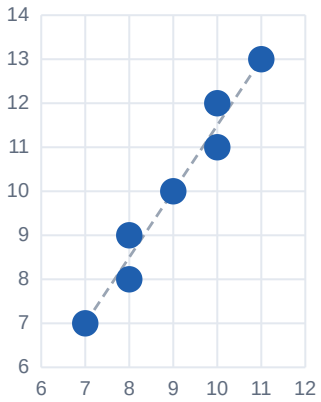
Graphics sec5 · p2 جه في

Q49 Using Bresenham's algorithm, digitize the line (7, 7) to (11, 13).

$|\Delta x| = |11 - 7| = |4|$, $|\Delta y| = |13 - 7| = |6|$ · $|\Delta y| > |\Delta x|$ · Swap x points with y points: X0 = 7, xend = 13 & y0 = 7, yend = 11 · $|\Delta x| = 6$, $|\Delta y| = 4$ · $p0 = 2\Delta y - \Delta x = 2$ · If $p_k < 0$: Setpixel($y_k + 1$, x_k), $p_{k+1} = p_k + 8$ · Otherwise: Setpixel($y_k + 1$, $x_k + 1$), $p_{k+1} = p_k - 4$

P	X	y	Setpixel(y, x)
-	7	7	(7,7)
2	8	8	(8,8)
-2	9	8	(8,9)
6	10	9	(9,10)
2	11	10	(10,11)
-2	12	10	(10,12)
6	13	11	(11,13)

Plotted pixels



Answer from: Graphics sec5 · p2

Graphics sec5 · p2 جه في

SECTION 05 · CIRCLE, ELLIPSE, ANTIALIASING

أسئلة Lec 5 05

Q50 **Explain the direct methods of drawing a circle (Pythagoras and trigonometry).**

- A circle is a set of all points in a plane that are all an equal distance from a single point, the center. The distance from a circle's center to a point on the circle is called the radius of the circle.
- A unit circle is the circle of radius 1 centered at the origin (0, 0) in the Cartesian coordinate system.
- Pythagoras: $x^2 + y^2 = r^2$; for a unit circle $x^2 + y^2 = 1 \rightarrow y = \sqrt{(1 - x^2)}$. We could draw a quarter circle by incrementing x from 0 to R in unit steps and solving for +y. For another circle: $y = yc + \sqrt{(r^2 - (x - xc)^2)}$. Method needs lots of computation, and gives non-uniform pixel spacing.
- Trigonometry: $\sin \theta = y/r$, $\cos \theta = x/r \rightarrow x = r \cos \theta$ and $y = r \sin \theta$; unit circle $x = \cos \theta$, $y = \sin \theta$; another circle: $y = yc + \sin \theta \cdot r$, $x = xc + \cos \theta \cdot r$.

Answer from: Lec 5 · p3–8

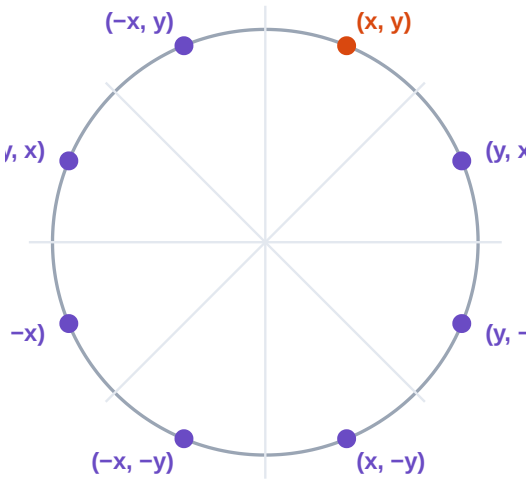
Lec 5 · p3–8 جه في

Q51 Explain the midpoint circle algorithm and the symmetry of a circle.

متكرر

- $R^2 = x^2 + y^2 \rightarrow x^2 + y^2 - R^2 = 0 \rightarrow$ decision parameter $P = x^2 + y^2 - R^2$: $P = 0$ on the circle, $P > 0$ outside the circle, $P < 0$ inside the circle.
- Symmetry of a circle: calculation of a circle point (x, y) in one octant yields the circle points shown for the other seven octants.
- Initial point (0, r) & $P0 = 1 - r$.
- If $p_k < 0$ the midpoint is inside the circle: $x_{k+1} = x_k + 1$, $y_{k+1} = y_k$, $P_{k+1} = P_k + 2x_{k+1} + 1$.
- If $p_k > 0$ the midpoint is outside the circle: $x_{k+1} = x_k + 1$, $y_{k+1} = y_k - 1$, $P_{k+1} = P_k + 2x_{k+1} + 1 - 2y_{k+1}$.
- Setpixel(x, y) → Setpixel(xc + x, yc + y) when the center is not (0, 0).

Eight-way symmetry of a circle



Answer from: Lec 5 · p9–13 · Graphics_Sec 6 · p1

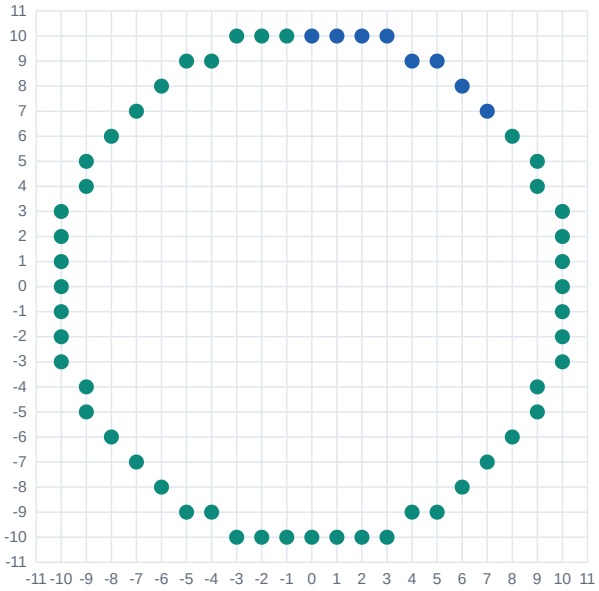
Graphics_Sec 6 · p1 Lec 5 · p9–13 جه في

Q52 **Using the midpoint circle algorithm, calculate the points of a circle with center (0, 0) and radius 10.**

Initial point (x0, y0) = (0, 10).

P	X	Y	$2x_{k+1}$	$2y_{k+1}$
-	0	10		
-9	1	10	2	20
-6	2	10	4	20
-1	3	10	6	20
6	4	9	8	18
-3	5	9	10	18
8	6	8	12	16
5	7	7	-	-

Octant points (blue) + eight-way symmetry (green)



Eight-way symmetry of (1, 10): Setpixel(1,10), (10,1), (10,−1), (1,−10), (−1,−10), (−10,−1), (−10,1), (−1,10).

Answer from: Lec 5 · p12–13

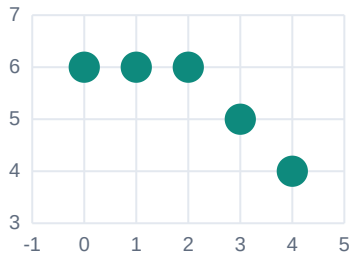
Lec 5 · p12–13 جه في

Q53 Using the mid-point circle drawing algorithm, draw a circle with a radius of 6 units, along the circle octant in the 1st quadrant.

Initial point (0, r) & P0 = 1 − r

p_k	x	Y
-	0	6
-5	1	6
-2	2	6
3	3	5
0	4	4

Octant points



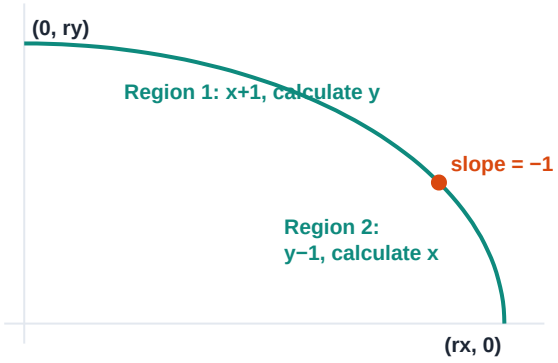
Answer from: Graphics_Sec 6 · p2

Graphics_Sec 6 · p2 جه في

Q54 Explain the midpoint ellipse algorithm (region 1 and region 2). متكرر

- Region 1: increment x, calculate y. Region 2: decrement y, calculate x.
- **Region 1:** initial point (x0, y0) = (0, r_y) & P0 = $r_y^2 - r_x^2 r_y + \frac{1}{4} r_x^2$. If $p_k < 0$ (inside): x + 1, y stays, $P_{k+1} = p_k + 2r_y^2 x_{k+1} + r_y^2$. If $p_k > 0$ (outside): x + 1, y − 1, $P_{k+1} = p_k + 2r_y^2 x_{k+1} - 2r_x^2 y_{k+1} + r_y^2$. Repeat until $2r_y^2 x \geq 2r_x^2 y$.
- **Region 2:** initial point = last point of region 1 & P0 = $r_y^2(x0 + \frac{1}{2})^2 + r_x^2(y0 - 1)^2 - r_x^2 r_y^2$. If $p_k < 0$ (inside): x + 1, y − 1, $P_{k+1} = p_k + 2r_y^2 x_{k+1} - 2r_x^2 y_{k+1} + r_x^2$. If $p_k > 0$ (outside): x stays, y − 1, $P_{k+1} = p_k - 2r_x^2 y_{k+1} + r_x^2$. Repeat until we reach point (r_x, 0).

Ellipse regions (first quadrant)



Answer from: Lec 5 · p15–21 · Graphics_Sec 6 · p1

Graphics_Sec 6 · p1 Lec 5 · p15–21 جه في

Q55 **Given input ellipse parameters rx = 8 and ry = 6, determine the raster positions along the ellipse path in the first quadrant.**

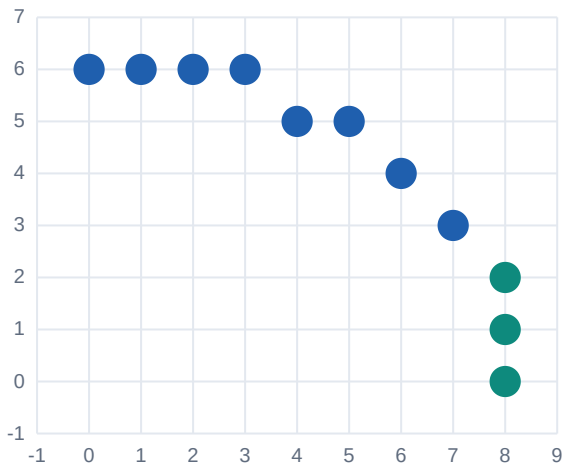
Region 1 · initial point (0, 6)

k	$p1_k$	(x_{k+1}, y_{k+1})	$2r_y^2x_{k+1}$	$2r_x^2y_{k+1}$
0	−332	(1, 6)	72	768
1	−224	(2, 6)	144	768
2	−44	(3, 6)	216	768
3	208	(4, 5)	288	640
4	−108	(5, 5)	360	640
5	288	(6, 4)	432	512
6	244	(7, 3)	504	384

Region 2 · initial point = last point of region 1 (7, 3)

k	$p2_k$	(x_{k+1}, y_{k+1})	$2r_y^2x_{k+1}$	$2r_x^2y_{k+1}$
0	−151	(8, 2)	576	256
1	233	(8, 1)	576	128
2	745	(8, 0)	—	—

Region 1 (blue) and region 2 (green)



Answer from: Lec 5 · p22–23

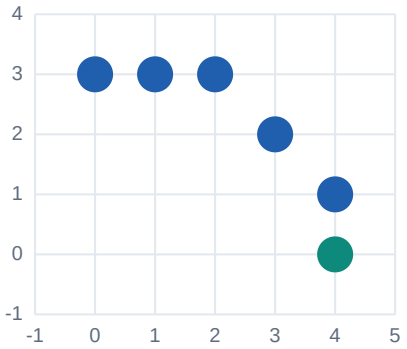
Lec 5 · p22–23 جه في

Q56 **Determine pixel positions along the ellipse path in the first quadrant using midpoint ellipse algorithm. Given: rx = 4 and ry = 3.**

Pk_region1	x	y	$2r_y^2x$	$2r_x^2y$
	0	3	0	96
−35	1	3	18	96
−8	2	3	36	96
37	3	2	54	64
36	4	1	72	32

Pk_region2	x	y	$2r_y^2x$	$2r_x^2y$
	4	1	72	32
38	4	0		

Region 1 (blue) and region 2 (green)



Answer from: Graphics_Sec 6 · p2

Graphics_Sec 6 · p2 جه في

Q57 What is aliasing? Explain antialiasing and super-sampling.

- Line segments and other graphics primitives generated by the raster algorithms have a jagged, or stair-step, appearance. This distortion of information due to low-frequency sampling (under sampling) is called **aliasing**.
- We can improve the appearance of displayed raster lines by applying **antialiasing** methods that compensate for the under sampling process.
- One way to increase sampling rate with raster systems is simply to display objects at higher resolution. However, even at the highest resolution possible with current technology, the jaggies will be apparent to some extent. There is a limit to how big we can make the frame buffer and still maintain the refresh rate at 60 frames or more per second.
- A straightforward antialiasing method is to increase sampling rate by treating the screen as if it were covered with a finer grid than is actually available, using multiple sample points across this finer grid to determine an appropriate intensity level for each screen pixel. This technique is called **super-sampling** (or post-filtering).
- Supersampling algorithms are often implemented by giving more weight to subpixels near the center of a pixel area.

Answer from: Lec 5 · p24–25

Lec 5 · p24–25 جه في

SECTION 06 · GEOMETRIC TRANSFORMATIONS

أسئلة Lec 6 06

Q58 **What are geometric transformations? What do linear transformations preserve?** متكرر

- Geometric transformation functions: translation, scaling, rotation, reflection, shearing.
- Geometric transformations map points in one space to points in another. These transformations can be linear or non-linear transformations.
- We'll focus on linear transformations that preserve co-linearity and ratios of distances (parallel lines remain parallel): maps origin to origin; **co-linearity:** all points lying on a line initially still lie on a line after transformation; **ratios of distances:** the midpoint of a line segment remains the midpoint after transformation.
- Linear transformation can be represented by (non-singular) matrices: [x′ y′] = M·[x y], M a square matrix of n columns and n rows; M = [a b; c d] → x′ = ax + by & y′ = cx + dy.

Answer from: Lec 6 · p2–5 · Graphics_Sec 7 · p1

Graphics_Sec 7 · p1 Lec 6 · p2–5 جه في

Q59 **Write the equations of 2D scaling, reflection, rotation, shear and translation.**

- **Scaling from the origin:** x′ = sx·x, y′ = sy·y; S = [sx 0; 0 sy]. Properties: doesn't preserve angles between lines except with uniform scaling sx = sy; the object must be centered on origin, or it will be shifted.
- **Reflection** creates a mirror image of an object across a line or point. Horizontal reflection: sy = −1 → x′ = x, y′ = −y. Vertical reflection: sx = −1 → x′ = −x, y′ = y. Diagonal reflection: sx = −1 and sy = −1 → x′ = −x, y′ = −y.
- **Rotation about the origin:** x = r cos φ, y = r sin φ, x′ = r cos(θ + φ), y′ = r sin(θ + φ) → **x′ = x cos θ − y sin θ, y′ = x sin θ + y cos θ**; R = [cos θ −sin θ; sin θ cos θ]; P′ = R·P.
- **Shear (skewing)** deforms shape like shifted slices. Shear in x: x′ = x + shx·y, y′ = y. Shear in y: x′ = x, y′ = shy·x + y.
- **Translation** over vector (tx, ty): x′ = x + tx, y′ = y + ty, or P′ = P + T.

Answer from: Lec 6 · p6–12

Lec 6 · p6–12 جه في

Q60 **Why do we use homogeneous coordinates? Write the 2D transformation matrices and their inverses.** متكرر

- Scale P′ = S·P, rotation P′ = R·P, shear P′ = Shr·P, translation P′ = T + P. We would like all transformations to be multiplications so we can concatenate them to combine transformations.
- Homogeneous coordinates: uniform representation of translation, rotation, scaling; uniform representation of points and vectors; compact representation of a sequence of transformations. Add an extra coordinate, W (weight = 1), to a point: P(x, y) → P(x, y, 1).
- Transformation matrices for 2D translation are now 3 × 3.

Transformation	2D Matrix	Inverse 2D Matrix
Translation	$\begin{vmatrix} 1 & 0 & tx \\ 0 & 1 & ty \\ 0 & 0 & 1 \end{vmatrix}$	$\begin{vmatrix} 1 & 0 & -tx \\ 0 & 1 & -ty \\ 0 & 0 & 1 \end{vmatrix}$
Scaling	$\begin{vmatrix} sx & 0 & 0 \\ 0 & sy & 0 \\ 0 & 0 & 1 \end{vmatrix}$	$\begin{vmatrix} 1/sx & 0 & 0 \\ 0 & 1/sy & 0 \\ 0 & 0 & 1 \end{vmatrix}$
Reflection	horizontal $\begin{vmatrix} 1 & 0 & 0 \\ 0 & -1 & 0 \\ 0 & 0 & 1 \end{vmatrix}$ vertical $\begin{vmatrix} -1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{vmatrix}$	horizontal $\begin{vmatrix} 1 & 0 & 0 \\ 0 & -1 & 0 \\ 0 & 0 & 1 \end{vmatrix}$ vertical $\begin{vmatrix} -1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{vmatrix}$
Rotation	$R(\theta) = \begin{vmatrix} \cos\theta & -\sin\theta & 0 \\ \sin\theta & \cos\theta & 0 \\ 0 & 0 & 1 \end{vmatrix}$	$R^T(\theta) = \begin{vmatrix} \cos\theta & \sin\theta & 0 \\ -\sin\theta & \cos\theta & 0 \\ 0 & 0 & 1 \end{vmatrix}$
Shearing	in y $\begin{vmatrix} 1 & 0 & 0 \\ shy & 1 & 0 \\ 0 & 0 & 1 \end{vmatrix}$ in x $\begin{vmatrix} 1 & shx & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{vmatrix}$	in y $\begin{vmatrix} 1 & 0 & 0 \\ 1/shy & 1 & 0 \\ 0 & 0 & 1 \end{vmatrix}$ in x $\begin{vmatrix} 1 & 1/shx & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{vmatrix}$

Inverse rotation matrices: $R^{-1}(\theta) = R(-\theta) = R^T(\theta)$.

Answer from: Lec 6 · p13–19 · Graphics_Sec 7 · p1

Graphics_Sec 7 · p1 Lec 6 · p13–19 جه في

Q61 State the concatenation properties of translation, scaling and rotation.

	Concatenation	Properties
Translation	$P'' = P \cdot T(tx1, ty1) \cdot T(tx2, ty2) = P \cdot T(tx1 + tx2, ty1 + ty2)$	1. $T(0, 0) = I$ · 2. $T(tx1, ty1) \cdot T(tx2, ty2) = T(tx1 + tx2, ty1 + ty2)$ · 3. $T^{-1}(tx1, ty1) = T(-tx1, -ty1)$
Scaling	$S(sx1, sy1) \cdot S(sx2, sy2) = S(sx1 \cdot sx2, sy1 \cdot sy2)$ — only diagonal elements in the matrix, easy to multiply	1. $S(1, 1) = I$ · 2. $S(sx1, sy1) \cdot S(sx2, sy2) = S(sx1 \cdot sx2, sy1 \cdot sy2)$ · 3. $S^{-1}(sx, sy) = S(1/sx, 1/sy)$
Rotation	$R(\theta) \cdot R(\varphi) = R(\theta + \varphi)$	$R(0) = I$ · $R(\theta) \cdot R(\varphi) = R(\theta + \varphi)$ · $R(\theta) \cdot R(\varphi) = R(\varphi) \cdot R(\theta)$ · inverse rotation $R^{-1}(\theta) = R^{T}(\theta)$

2D polygon transformation: translate polygon — apply the same operation on all points. Works always, for all transformations of objects defined as a set of points.

Answer from: Lec 6 · p20–28

Lec 6 · p20–28 جه في

Q62 How do you rotate or scale an object with respect to a point that is not the origin?

- Combine transformations of different type: translate, rotate, translate; translate, scale, translate; translate, reflect, translate. Use to rotate or scale an object w.r.t. a point that is not the origin. Implement by multiplication of the corresponding homogeneous matrices.
- **Rotation around a pivot point:** 1. Translate the object so that the pivot point moves to the origin. 2. Rotate around origin. 3. Translate the object so that the pivot point is back to its original position. **$P' = T(tx, ty) \cdot R(\theta) \cdot T(-tx, -ty) \cdot p$**.
- **Scale around a pivot point:** same three steps → **$P' = T(tx, ty) \cdot S(sx, sy) \cdot T(-tx, -ty) \cdot p$**.

Answer from: Lec 6 · p29–31

Lec 6 · p29–31 جه في

Q63 Explain transformations in 3D and write their matrices and inverses. متكرر

- Just add one more coordinate/axis; a point is represented as [x y z]. Remember to use homogeneous coordinates: embed scale and rotation matrices as upper-left sub-matrices and translation vectors as upper-right sub-vectors of the right column [x y z 1].
- Scaling looks just like the 2D version; we just added an sz term. Translation: similar to the 2D version, just with one more entry dz.
- Reflection in 3D: reflection over planes.
- Rotation in 3D: need to specify which axis the rotation is about; counterclockwise when looking along the positive half towards the origin.
- Inverses and composition in 3D: composition works exactly the same way.
- Scale an object w.r.t. a fixed position (xf, yf, zf): 1. translate the fixed point to the origin; 2. apply the scaling transformation relative to the coordinate origin; 3. translate the fixed point back to its original position.

Transformation	3D Matrix	Inverse 3D Matrix
Translation	$\begin{vmatrix} 1 & 0 & 0 & tx \\ 0 & 1 & 0 & ty \\ 0 & 0 & 1 & tz \\ 0 & 0 & 0 & 1 \end{vmatrix}$	$\begin{vmatrix} 1 & 0 & 0 & -tx \\ 0 & 1 & 0 & -ty \\ 0 & 0 & 1 & -tz \\ 0 & 0 & 0 & 1 \end{vmatrix}$
Scaling	$\begin{vmatrix} sx & 0 & 0 & 0 \\ 0 & sy & 0 & 0 \\ 0 & 0 & sz & 0 \\ 0 & 0 & 0 & 1 \end{vmatrix}$	$\begin{vmatrix} 1/sx & 0 & 0 & 0 \\ 0 & 1/sy & 0 & 0 \\ 0 & 0 & 1/sz & 0 \\ 0 & 0 & 0 & 1 \end{vmatrix}$
Reflection	yz plane $\begin{vmatrix} -1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{vmatrix}$ xz plane $\begin{vmatrix} 1 & 0 & 0 & 0 \\ 0 & -1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{vmatrix}$ xy plane $\begin{vmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & -1 & 0 \\ 0 & 0 & 0 & 1 \end{vmatrix}$	the same matrices

Rotation

$$Rx(\theta)=\begin{vmatrix}1 & 0 & 0 & 0\\ 0 & \cos\theta & -\sin\theta & 0\\ 0 & \sin\theta & \cos\theta & 0\\ 0 & 0 & 0 & 1\end{vmatrix} \qquad Rx^{T}(\theta)=\begin{vmatrix}1 & 0 & 0 & 0\\ 0 & \cos\theta & \sin\theta & 0\\ 0 & -\sin\theta & \cos\theta & 0\\ 0 & 0 & 0 & 1\end{vmatrix}$$

$$Ry(\theta)=\begin{vmatrix}\cos\theta & 0 & \sin\theta & 0\\ 0 & 1 & 0 & 0\\ -\sin\theta & 0 & \cos\theta & 0\\ 0 & 0 & 0 & 1\end{vmatrix} \qquad Ry^{T}(\theta)=\begin{vmatrix}\cos\theta & 0 & -\sin\theta & 0\\ 0 & 1 & 0 & 0\\ \sin\theta & 0 & \cos\theta & 0\\ 0 & 0 & 0 & 1\end{vmatrix}$$

$$Rz(\theta)=\begin{vmatrix}\cos\theta & -\sin\theta & 0 & 0\\ \sin\theta & \cos\theta & 0 & 0\\ 0 & 0 & 1 & 0\\ 0 & 0 & 0 & 1\end{vmatrix} \qquad Rz^{T}(\theta)=\begin{vmatrix}\cos\theta & \sin\theta & 0 & 0\\ -\sin\theta & \cos\theta & 0 & 0\\ 0 & 0 & 1 & 0\\ 0 & 0 & 0 & 1\end{vmatrix}$$

Answer from: Lec 6 · p37–42 · Graphics_Sec 7 · p2

Graphics_Sec 7 · p2 | Lec 6 · p37–42 | جه في

Q64 **Rotate point p by angle 50 degree, then scale it to the double horizontally, then translate it 10 points to the bottom.**

P′ = R(50)·P

P′ = S(2, 1)·R(50)·p

P′ = T(0, −10)·S(2, 1)·R(50)·p

$$\begin{vmatrix}x'\\ y'\\ 1\end{vmatrix}=\begin{vmatrix}1 & 0 & 0\\ 0 & 1 & -10\\ 0 & 0 & 1\end{vmatrix}\cdot\begin{vmatrix}2 & 0 & 0\\ 0 & 1 & 0\\ 0 & 0 & 1\end{vmatrix}\cdot\begin{vmatrix}\cos 50 & -\sin 50 & 0\\ \sin 50 & \cos 50 & 0\\ 0 & 0 & 1\end{vmatrix}\cdot\begin{vmatrix}x\\ y\\ 1\end{vmatrix}$$

Answer from: Lec 6 · p32

Lec 6 · p32 | جه في

Q65 Rotate point p by angle 40 degree around point (3, 4).

P′ = T(−3, −4)·p

P′ = R(40)·T(−3, −4)·P

P′ = T(3, 4)·R(40)·T(−3, −4)·P

$$\begin{vmatrix} x' \\ y' \\ 1 \end{vmatrix} = \begin{vmatrix} 1 & 0 & 3 \\ 0 & 1 & 4 \\ 0 & 0 & 1 \end{vmatrix} \cdot \begin{vmatrix} \cos 40 & -\sin 40 & 0 \\ \sin 40 & \cos 40 & 0 \\ 0 & 0 & 1 \end{vmatrix} \cdot \begin{vmatrix} 1 & 0 & -3 \\ 0 & 1 & -4 \\ 0 & 0 & 1 \end{vmatrix} \cdot \begin{vmatrix} x \\ y \\ 1 \end{vmatrix}$$

Answer from: Lec 6 · p33

Lec 6 · p33 جه في

Q66 Perform a rotation by angle 90 degree of a triangle with vertices A(0, 0), B(2, 1) and C(5, 4): about origin, about the point (−2, −1), about the triangle centroid.

- **About origin:** PA′ = R(90)·A, PB′ = R(90)·B, PC′ = R(90)·C.
- **About point (−2, −1):** PA′ = T(−2, −1)·R(90)·T(2, 1)·A (the same for B and C).
- **About the triangle centroid:** triangle centroid = ((0 + 2 + 5)/3, (0 + 1 + 4)/3) = (2.33, 1.66) → PA′ = T(2.33, 1.66)·R(90)·T(−2.33, −1.66)·A (the same for B and C).

$$\mathbf{PB'} = \begin{vmatrix} 1 & 0 & -2 \\ 0 & 1 & -1 \\ 0 & 0 & 1 \end{vmatrix} \cdot \begin{vmatrix} \cos 90 & -\sin 90 & 0 \\ \sin 90 & \cos 90 & 0 \\ 0 & 0 & 1 \end{vmatrix} \cdot \begin{vmatrix} 1 & 0 & 2 \\ 0 & 1 & 1 \\ 0 & 0 & 1 \end{vmatrix} \cdot \begin{vmatrix} 2 \\ 1 \\ 1 \end{vmatrix}$$

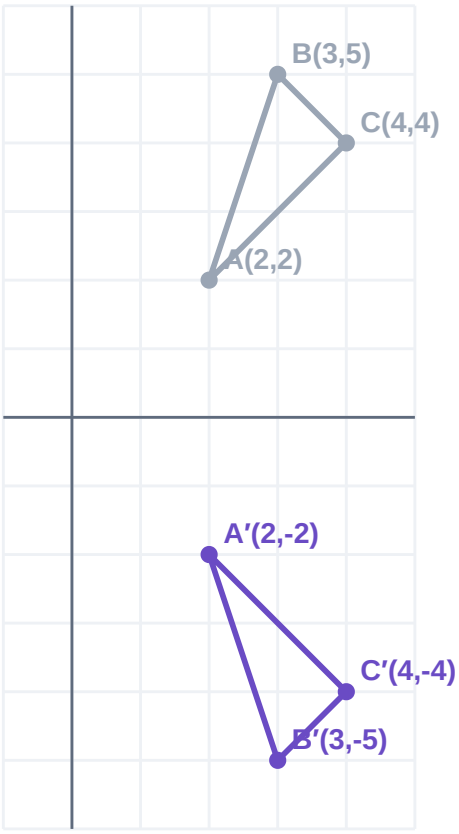
Answer from: Lec 6 · p34–36

Lec 6 · p34–36 جه في

Q67 **A triangle is defined by three points A(2, 2), B(3, 5), C(4, 4): what's the transformation coordinate after horizontal reflection?**

Answer: **A(2, −2), B(3, −5), C(4, −4)**

Before (gray) and after horizontal reflection (purple)



Answer from: Graphics_Sec 7 · p3

Graphics_Sec 7 · p3 جه في

Q68 **Rotate point p by angle 50 degree, then rotate it by angle −60, then scale it to half vertically.**

$$\begin{vmatrix} x' \\ y' \\ 1 \end{vmatrix} = \begin{vmatrix} 1 & 0 & 0 \\ 0 & 0.5 & 0 \\ 0 & 0 & 1 \end{vmatrix} \cdot \begin{vmatrix} \cos -10 & -\sin -10 & 0 \\ \sin -10 & \cos -10 & 0 \\ 0 & 0 & 1 \end{vmatrix} \cdot \begin{vmatrix} x \\ y \\ 1 \end{vmatrix}$$

Answer from: Graphics_Sec 7 · p3

Graphics_Sec 7 · p3 جه في

Q69 **Rotate point p by angle 20 around point (5, 6), then rotate angle by −50 around point (3, 7).**

$$\begin{vmatrix} x' \\ y' \\ 1 \end{vmatrix} = \begin{vmatrix} 1 & 0 & 3 \\ 0 & 1 & 4 \\ 0 & 0 & 1 \end{vmatrix} \cdot \begin{vmatrix} \cos 40 & -\sin 40 & 0 \\ \sin 40 & \cos 40 & 0 \\ 0 & 0 & 1 \end{vmatrix} \cdot \begin{vmatrix} 0 & 0 & 2 \\ 0 & 0 & -1 \\ 0 & 0 & 1 \end{vmatrix} \cdot \begin{vmatrix} \cos 20 & -\sin 20 & 0 \\ \sin 20 & \cos 20 & 0 \\ 0 & 0 & 1 \end{vmatrix} \cdot \begin{vmatrix} 0 & 0 & -5 \\ 0 & 0 & -6 \\ 0 & 0 & 1 \end{vmatrix} \cdot \begin{vmatrix} x \\ y \\ 1 \end{vmatrix}$$

(Answer copied exactly as written in Graphics_Sec 7.)

Answer from: Graphics_Sec 7 · p3

Graphics_Sec 7 · p3 جه في

Q70 **Magnify a triangle with vertices A(1, 1), B(2, 3) and C(7, 3) to twice its size while keeping C(7, 3) fixed.**

$$PA' = \begin{vmatrix} 1 & 0 & 7 \\ 0 & 1 & 3 \\ 0 & 0 & 1 \end{vmatrix} \cdot \begin{vmatrix} 2 & 0 & 0 \\ 0 & 2 & 0 \\ 0 & 0 & 1 \end{vmatrix} \cdot \begin{vmatrix} 1 & 0 & -7 \\ 0 & 1 & -3 \\ 0 & 0 & 1 \end{vmatrix} \cdot \begin{vmatrix} 1 \\ 1 \\ 1 \end{vmatrix}$$

$$PB' = \begin{vmatrix} 1 & 0 & 7 \\ 0 & 1 & 3 \\ 0 & 0 & 1 \end{vmatrix} \cdot \begin{vmatrix} 2 & 0 & 0 \\ 0 & 2 & 0 \\ 0 & 0 & 1 \end{vmatrix} \cdot \begin{vmatrix} 1 & 0 & -7 \\ 0 & 1 & -3 \\ 0 & 0 & 1 \end{vmatrix} \cdot \begin{vmatrix} 2 \\ 3 \\ 1 \end{vmatrix}$$

$$PC' = \begin{vmatrix} 1 & 0 & 7 \\ 0 & 1 & 3 \\ 0 & 0 & 1 \end{vmatrix} \cdot \begin{vmatrix} 2 & 0 & 0 \\ 0 & 2 & 0 \\ 0 & 0 & 1 \end{vmatrix} \cdot \begin{vmatrix} 1 & 0 & -7 \\ 0 & 1 & -3 \\ 0 & 0 & 1 \end{vmatrix} \cdot \begin{vmatrix} 7 \\ 3 \\ 1 \end{vmatrix}$$

Answer from: Graphics_Sec 7 · p3

Graphics_Sec 7 · p3 جه في

Q71 **For a pyramid centred at (5, 2, 1): scale in object space by 2 in the x, y, and 3 in the z; rotate clockwise in object's space by 10° around x axis, 20° around y, and 70° around z; translate by (1, 7, 3) in world space.**

$$\begin{vmatrix} 1 & 0 & 0 & 6 \\ 0 & 1 & 0 & 9 \\ 0 & 0 & 1 & 4 \\ 0 & 0 & 0 & 1 \end{vmatrix} \cdot Rz(70) \cdot Ry(20) \cdot Rx(10) \cdot \begin{vmatrix} 2 & 0 & 0 & 0 \\ 0 & 2 & 0 & 0 \\ 0 & 0 & 3 & 0 \\ 0 & 0 & 0 & 1 \end{vmatrix} \cdot \begin{vmatrix} 1 & 0 & 0 & -5 \\ 0 & 1 & 0 & -2 \\ 0 & 0 & 1 & -1 \\ 0 & 0 & 0 & 1 \end{vmatrix} \cdot \begin{vmatrix} x \\ y \\ z \\ 1 \end{vmatrix}$$

Answer from: Graphics_Sec 7 · p3

Graphics_Sec 7 · p3 جه في

SECTION 07 · OPENGL / GLUT CODE FROM THE SLIDES

07 أسئلة الأكواد

Q72 **Write the main and myInit functions of an OpenGL program with a 400 × 300 display window and gluOrtho2D(−200, 200, −150, 150).**

```
int main(int argc, char** argv)
{
    glutInit(&argc, argv);
    glutInitDisplayMode(GLUT_RGB);//set display mode
    glutInitWindowSize(400, 300);//set window size
    glutInitWindowPosition(100, 150);//set window position
    glutCreateWindow("graphics");
    myInit();
    glutDisplayFunc(myDisplay);
    glutReshapeFunc(reshape);
    Timer(0);
    glutMainLoop();
    return 0;
}
void myInit()
{
    glClearColor(1.0, 1.0, 1.0, 0.0);
    glColor3f(0.0f, 0.0f, 0.0f);
    glMatrixMode(GL_PROJECTION);
    glLoadIdentity();
    gluOrtho2D(-200, 200, -150, 150);
}
```

Answer from: Lec 3 · p10

Lec 3 · p10 جه في

Q73 Write myDisplay to draw a blue triangle with vertices (−200, 10), (10, 10), (10, 150).

```
void myDisplay()
{
    glClear(GL_COLOR_BUFFER_BIT); // if it doesn't written there will be no display
    glPointSize(5);
    glColor3f(0.0f, 0.0f, 1.0f);
    glBegin(GL_TRIANGLES);
    glVertex2f(-200, 10);
    glVertex2f(10, 10);
    glVertex2f(10, 150);
    glEnd();
    glFlush();
}
```

Answer from: Lec 3 · p11

Lec 3 · p11 جه في

Q74 Can you draw a simple figure without gluOrtho2D? Write the code.

```
void myInit()
{
    glClearColor(1.0, 1.0, 1.0, 0.0);
    glColor3f(0.0f, 0.0f, 0.0f);
    glMatrixMode(GL_PROJECTION);
    glLoadIdentity();
    //gluOrtho2D(-100,100,-100,100);
}
void myDisplay()
{
    glClear(GL_COLOR_BUFFER_BIT);
    glPointSize(5);
    glColor3f(0.0, 0.0, 1.0);
    glBegin(GL_TRIANGLES);
    glVertex2f(-0.5, 0);
    glVertex2f(0, 0.5);
    glVertex2f(0.5, 0);
    glEnd();
    glFlush();
}
```

Answer from: Lec 3 · p17

Lec 3 · p17 جه في

Q75 **Write code that creates multiple viewports where each viewport has a scene.**

```
void myDisplay()
{
    glClear(GL_COLOR_BUFFER_BIT);
    glPointSize(5);
    glViewport(0, 0, 100, 100);
    glBegin(GL_TRIANGLES);
    glVertex2f(-100, -100);
    glVertex2f(100, -100);
    glVertex2f(0, 150);
    glEnd();

    glViewport(100, 100, 100, 100);
    glBegin(GL_TRIANGLES);
    glVertex2f(-100, -100);
    glVertex2f(100, -100);
    glVertex2f(0, 150);
    glEnd();

    glViewport(200, 200, 100, 100);
    glBegin(GL_TRIANGLES);
    glVertex2f(-100, -100);
    glVertex2f(100, -100);
    glVertex2f(0, 150);
    glEnd();

    glFlush();
}
```

Answer from: Lec 3 · p18

Lec 3 · p18 جه في

Q76 Write the lineDDA function.

```
inline int round (const float a) { return int (a + 0.5); }
void lineDDA (int x0, int y0, int xEnd, int yEnd)
{
    int dx = xEnd - x0, dy = yEnd - y0, steps, k;
    float xIncrement, yIncrement, x = x0, y = y0;
    if (fabs (dx) > fabs (dy))
        steps = fabs (dx);
    else
        steps = fabs (dy);
    xIncrement = float (dx) / float (steps); // xIncrement =1
    yIncrement = float (dy) / float (steps); //yIncrement =dy/dx=m
    setPixel (round (x), round (y));
    for (k = 0; k < steps; k++) {
        x += xIncrement;
        //x=x+1
        y += yIncrement;
        //y=y+m
        setPixel (round (x), round (y));
    }
}
```

Answer from: Lec 4 · p18

Lec 4 · p18 جه في

Q77 Write the midpoint circle algorithm code.

```
void myDisplay()
{
    glClear(GL_COLOR_BUFFER_BIT); // if it doesn't written there will be no display
    glPointSize(1);
    circleMidpoint(0, 0, 100,GL_POINTS);
    glFlush();
}
void circleMidpoint(int xc, int yc, int radius, GLenum mode)
{
    int x = 0;
    int y = radius;
    int p0 = 1 - radius;
    circleplot(xc, yc, x, y, mode);
    while (x < y)
    {
        x++;
        if (p0 < 0)
            p0 = p0 + 2 * x + 1;
        else
        {
            y--;
            p0 = p0 + 2 * x -2*y + 1;
        }
        circleplot(xc, yc, x, y, mode);
    }
}
void circleplot(int xc, int yc, int x, int y, GLenum mode)
{
    glBegin(mode);
    glVertex2f(xc + x, yc + y);
    glVertex2f(xc - x, yc + y);
    glVertex2f(xc + x, yc - y);
    glVertex2f(xc - x, yc - y);
    glVertex2f(xc + y, yc + x);
    glVertex2f(xc - y, yc + x);
    glVertex2f(xc + y, yc - x);
    glVertex2f(xc - y, yc - x);
    glEnd();
}
```

Answer from: Lec 5 · p14

Lec 5 · p14 جه في